VINUNIVERSITY

Function Calling & MCP

Phân biệt Function Calling (khả năng của model) và Model Context Protocol (giao thức kết nối) — qua code chạy được, từ demo tới production.

Giảng viên: Nguyễn Hải Trường

https://truongnh-gde.dev

AGENDA

- Phần 1 — Function Calling
- Phần 2 — MCP Basics
- Phần 3 — Production
- Phần 4 — Thực chiến

LEARNING OUTCOMES

Sau buổi này bạn sẽ làm được

1

Giải thích được sự khác nhau

Nói rõ Function Calling là khả năng của model, MCP là giao thức — và vì sao chúng bổ sung nhau chứ không thay thế.

2

Đọc hiểu code cả hai cách

Nhìn vào một file biết ngay: schema ở đâu, ai thực thi tool, tool được khám phá lúc nào.

3

Tự viết một MCP server

Thêm tool mới bằng @mcp.tool(), chạy client kiểm tra, đăng ký vào Claude Code / Gemini CLI.

4

Biết cần gì để lên production

Auth bằng bearer token, Tool Registry để agent tự tìm tool, versioning để không phá client cũ.

Model rất giỏi ngôn ngữ, nhưng không biết gì về thế giới thật

Nó không biết thời tiết hôm nay, không đọc được database của bạn, không gọi được API nội bộ.

KHÔNG CÓ CÔNG CỤ

User:

“Thời tiết Hà Nội hôm nay thế nào?”

LLM

“Tôi không có dữ liệu thời tiết thời gian thực.”

Model bó tay — hoặc tệ hơn: nó đoán và nói sai (hallucination).

CÓ CÔNG CỤ (TOOL)

User:

“Thời tiết Hà Nội hôm nay thế nào?”

`LLM: "gọi get_weather(city='Hà Nội')"`

Hệ thống chạy tool → “29°C, mưa nhẹ”

BIG PICTURE

Hai khái niệm — hai tầng khác nhau

Đây là nguồn nhầm lẫn số 1. Chúng không cạnh tranh: MCP dùng Function Calling ở bên dưới.

TẦNG MODEL

Function Calling

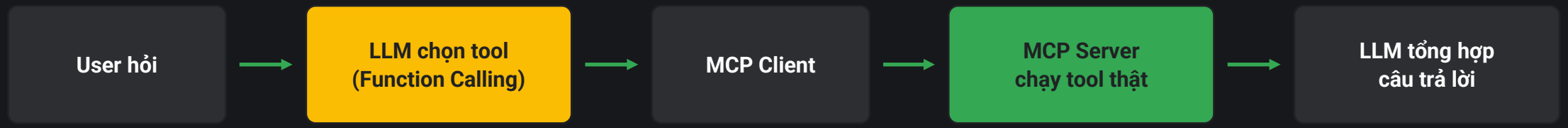
Một khả năng của model (model capability).
Model được huấn luyện để: nhìn danh sách tool → chọn tool → sinh JSON tham số.

TẦNG HẠ TẦNG

MCP

Một giao thức chuẩn (protocol) — như USB-C hay HTTP cho thế giới AI.
Định nghĩa cách Client kết nối Server để khám phá & dùng tool.

Chúng lắp vào nhau như thế nào



Function Calling = bước "LLM chọn tool" · MCP = đường ống nối Client ↔ Server (khám phá + gọi)

PHẦN 1

Function Calling

Model quyết định gọi tool nào — nhưng app mới là nơi chạy.

Code mẫu: 01-function-calling/weather_function_calling.py

Model KHÔNG chạy hàm.

Nó chỉ nói “hãy gọi hàm X với tham số Y”.

Mọi việc thực thi — gọi API, đọc DB, ghi file — do CODE CỦA BẠN làm.

Model không có quyền truy cập gì cả.

CHI TIẾT

Vòng đời một lượt Function Calling — bước 1 & 2

Câu hỏi ví dụ: "Thời tiết Hà Nội và Đà Nẵng hôm nay thế nào?" — chú ý model sẽ gọi tool 2 lần.

1 App chuẩn bị "hộp công cụ"

Developer viết schema BẰNG TAY — phải khớp chính xác với hàm Python thật.

Tool: `get_weather`

Mô tả: "Lấy thời tiết hiện tại của một thành phố"

Tham số: `city : string (bắt buộc)`

⚠ 15+ dòng boilerplate cho MỘT tool.

⚠ Sửa hàm mà quên sửa schema → model gọi sai, lỗi lúc runtime.

Nhược điểm này là lý do MCP ra đời.

2 Gửi prompt + schema cho model

Mỗi lượt gọi API đều phải gửi lại toàn bộ danh sách tool.



`"Thời tiết HN và ĐN?"`
`+ schema get_weather`

Model suy luận (bên trong):

"Câu hỏi có 2 thành phố. Tool này nhận 1 city mỗi lần → tôi cần gọi nó 2 lần, song song."

Bước 3 → 5: model ra chỉ thị, app thực thi, model tổng hợp

3 **Model TRẢ VỀ yêu cầu gọi tool — không tự chạy**

Phản hồi của model không phải câu văn, mà là một mảng function_calls.

```
function_calls: [
  get_weather(city="Hà Nội"),
  get_weather(city="Đà Nẵng"),
]
```

4 **APP tự thi hành hàm get_weather**

Đây là chỗ code của bạn chạy thật: gọi API thời tiết, đọc DB, bất cứ gì.

```
APP CHẠY, KHÔNG PHẢI MODEL
get_weather("Hà Nội")    -> "29°C, mưa nhẹ"
get_weather("Đà Nẵng")   -> "30°C, nhiều mây"
```

5 **Gửi kết quả lại — model viết câu trả lời cuối**

Model nhận dữ liệu thô, biến thành văn tự nhiên theo system instruction.

"Hà Nội 29°C, mưa nhẹ 🌧 nhớ mang ô nhé!
Đà Nẵng 30°C, nhiều mây 🌤 dễ chịu!"

Bước 3 → 4 → 5 có thể lặp nhiều vòng: model gọi tool, xem kết quả, rồi gọi tool khác. Đó là "agent loop".

Nhìn code thật — phần 1: schema viết tay

PYTHON

```python
from google import genai
from google.genai import types

# App phải TỰ MÔ TẢ tool cho model — viết tay, dễ sai
get_weather_declaration = types.FunctionDeclaration(
    name="get_weather",
    description="Lấy thời tiết hiện tại của một thành phố",
    parameters=types.Schema(
        type=types.Type.OBJECT,
        properties={
            "city": types.Schema(
                type=types.Type.STRING,
                description="Tên thành phố",
            )
        },
        required=["city"],
    ),
)

TOOLS = [types.Tool(function_declarations=[get_weather_declaration])]
```

Đọc slide này thế nào

- name — model dùng đúng chuỗi này để gọi. Sai một chữ là lỗi.
- description — quan trọng nhất! Đây là "lời quảng cáo" để model biết khi nào nên dùng tool.
- parameters — kiểu dữ liệu từng tham số, theo chuẩn JSON Schema.
- required — thiếu thì model có thể bỏ trống tham số.

Ghi nhớ

20 dòng code này chỉ để MÔ TẢ một hàm 3 dòng. Và nó chỉ dùng được với Gemini.

Nhìn code thật — phần 2 & 3: hàm thực thi và vòng lặp

Phần 2 — hàm Python thật (model không bao giờ chạy những dòng này)

```
def get_weather(city: str) -> str:
    """Trả về thời tiết (mock) của city."""
    mock_data = {
        "Hà Nội":  {"nhiệt_độ": "29°C", "thời_tiết": "trời mưa nhẹ", ...},
        "Đà Nẵng": {"nhiệt_độ": "30°C", "thời_tiết": "nhiều mây", ...},
    }
    return json.dumps({"city": city, **mock_data.get(city, default)})
```

Phần 3 — vòng lặp: nhận yêu cầu → chạy → trả lại

```
while resp.function_calls:                    # model còn muốn gọi tool?
    contents.append(resp.candidates[0].content)

    for fc in resp.function_calls:
        result = get_weather(**fc.args)        # <-- APP chạy, không phải model
        function_responses.append(
            types.Part.from_function_response(
                name=fc.name, response={"result": result}))

    contents.append(types.Content(role="user", parts=function_responses))
    resp = client.models.generate_content(...) # gọi model lần nữa
```

LIVE DEMO · 5 PHÚT

Demo 01 — chạy thật

TERMINAL

```
$ export GEMINI_API_KEY=...
$ cd 01-function-calling && python weather_function_calling.py
```

OUTPUT

```
User: Thời tiết Hà Nội và Đà Nẵng hôm nay thế nào?

  [model yêu cầu] get_weather({'city': 'Hà Nội'})
  [app thực thi]  -> {"city": "Hà Nội", "nhiệt_độ": "29°C", ...}
  [model yêu cầu] get_weather({'city': 'Đà Nẵng'})
  [app thực thi]  -> {"city": "Đà Nẵng", "nhiệt_độ": "30°C", ...}

Trả lời: Hà Nội 29°C, mưa nhẹ - nhớ mang ô nhé!
         Đà Nẵng 30°C, nhiều mây - thời tiết dễ chịu!
```

Chỉ cho lớp thấy

- Hai dòng “[model yêu cầu]” — model chỉ ra chỉ thị.
- Hai dòng “[app thực thi]” — code mình chạy.
- Câu trả lời cuối có emoji và lời khuyên vì system_instruction yêu cầu vậy.

Thử ngay tại lớp: Đổi câu hỏi thành “Tôi nên mang gì khi ra ngoài ở Hà Nội?” — model vẫn gọi get_weather, dù câu hỏi không hề có chữ “thời tiết”. Đó là sức mạnh của description.

VÌ SAO CẦN MCP

Ba nhược điểm khiến Function Calling thuần khó mở rộng

Schema viết tay

FunctionDeclaration dài 15–20 dòng cho mỗi tool. Sửa hàm mà quên sửa schema → model gọi sai tham số, lỗi chỉ hiện ra lúc runtime.

Hai nguồn sự thật cho cùng một thứ.

Tool bị khoá trong app

App A có get_weather. App B muốn dùng? Copy cả schema lẫn hàm. Sửa logic ở A phải nhớ sửa cả B, C, D.

Không có cách đóng gói để chia sẻ.

Mỗi provider một format

```
Google: FunctionDeclaration(...)
OpenAI: {"type":"function",...}
Anthropic: {"input_schema":...}
```

Đổi model = viết lại toàn bộ schema.

MCP giải quyết cả ba — bằng cách tách tool ra khỏi app và chuẩn hoá phương thức.

PHẦN 2

MCP Basics



Tool được tách ra một server độc lập, tự công bố chính mình.
Code mẫu: 02-mcp-basics/weather_server.py + weather_client.py

40 phút

MCP công bố ba loại tài nguyên

HÀNH ĐỘNG

Tools

Việc có tác động: gửi email, ghi DB, gọi API. Model quyết định gọi.

```
get_weather(city)
```

DỮ LIỆU ĐỌC

Resources

Nội dung để đọc, định danh bằng URI. Giống GET, không tác dụng phụ.

```
server://info
```

MẪU CÂU LỆNH

Prompts

Template prompt server gợi ý sẵn cho user chọn, ví dụ “review code này”.

```
/review-pr
```

Trong repo này: 02 và 03 dùng Tools; versioned_server.py còn công bố thêm một Resource là server://info.

Ba vai trong kiến trúc MCP

`Host`

Ứng dụng người dùng thấy: Claude Desktop, Cursor, IDE.

`Client`

Lớp trong Host giữ 1 kết nối tới 1 server. N server = N client.

`Server`

Tiến trình chứa & chạy tool. Có thể ở local (stdio) hay từ xa (HTTP).

API và MCP — khác nhau ở chỗ ai là người gọi

Nhầm lẫn số 2: "MCP có phải API kiểu mới không?" — Không. REST API viết cho lập trình viên đọc; MCP viết cho model đọc.

CHO CON NGƯỜI

REST API

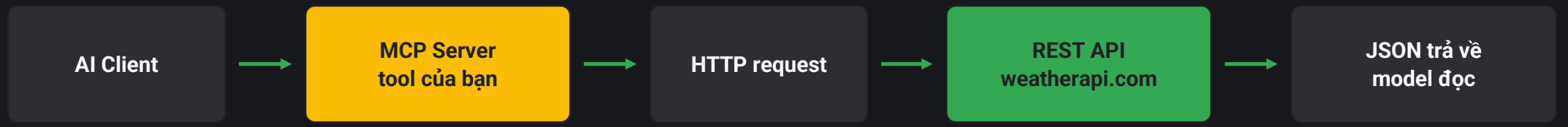
Hợp đồng giữa hai đoạn code. Con người đọc tài liệu (OpenAPI/Swagger) rồi viết client.
Endpoint, method, tham số đều hard-code lúc viết. Đổi API = sửa code, build lại, deploy lại.

CHO MODEL

MCP

Hợp đồng giữa một server và một model. Server tự công bố mình: list_tools() trả về tên, mô tả, schema.
Model đọc mô tả lúc chạy rồi tự chọn tool. Thêm tool mới, client không sửa một dòng.

MCP không thay thế API — nó bọc lấy API



API = hợp đồng cho code · MCP = hợp đồng cho model · Một MCP server thường chỉ là lớp vỏ mỏng bọc quanh REST API đã có

API và MCP — so sánh theo từng tiêu chí

	REST API	MCP
Ai là người gọi?	Lập trình viên viết code gọi	Model tự quyết định gọi lúc chạy
Biết có gì để gọi	Đọc tài liệu, hard-code endpoint	list_tools() hỏi server lúc chạy
Thêm chức năng mới	**Client phải sửa code và deploy lại**	**Server thêm @mcp.tool(), client giữ nguyên**
Mô tả tham số	OpenAPI/Swagger — cho người đọc	JSON Schema + docstring — cho model đọc
Trạng thái phiên	Stateless, mỗi request độc lập	Có phiên, hai chiều (sampling, progress)
Đổi nhà cung cấp	Viết lại client cho API mới	Đổi server, client dùng chung một chuẩn

Vẫn dùng REST API khi

- Bên gọi là code bạn viết, biết trước cần gọi gì.
- Cần throughput cao, latency thấp, hợp đồng ổn định.
- Không có model nào đứng ở đầu bên kia.

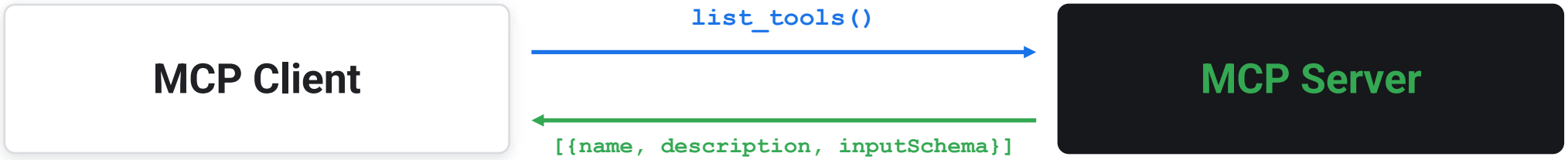
Chuyển sang MCP khi

- Bên gọi là model và bạn muốn nó tự chọn tool.
- Cùng một tool phải dùng lại ở nhiều app, nhiều model.
- Tool thay đổi thường xuyên mà không muốn deploy lại client.

LIST_TOOLS()

Bước 1 — KHÁM PHÁ: “Anh có tool gì?”

Client không hard-code danh sách tool. Nó hỏi server lúc chạy, và server tự trả lời.



Schema được SINH TỰ ĐỘNG từ:

`city: str`	→	type: "string", required
`-> str`	→	kiểu trả về
`docstring`	→	description cho model đọc
`@mcp.tool()`	→	đăng ký vào danh sách công bố

Một nguồn sự thật duy nhất: chính hàm Python.

SERVER KHAI BÁO — 4 DÒNG

```python
@mcp.tool()
def get_weather(city: str) -> str:
    """Lấy thời tiết hiện tại của một thành phố."""
    return f"{city}: {_MOCK_DB.get(city)}"
```

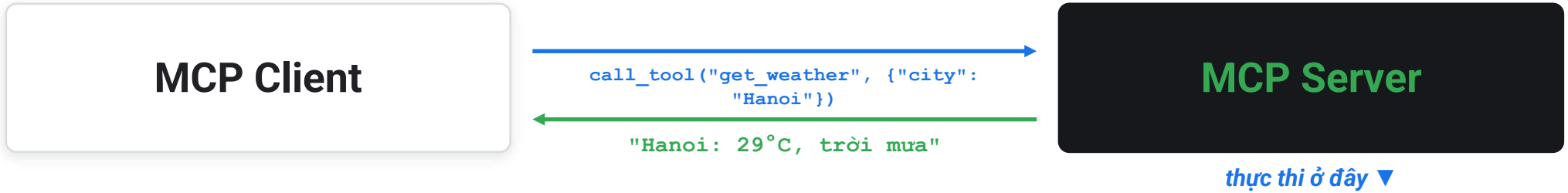
So với Function Calling

FC: developer viết schema thủ công 15–20 dòng
MCP: @mcp.tool() → schema tự sinh, không lệch được

CALL_TOOL()

Bước 2 — GỌI TOOL: “Cho tôi thời tiết Hà Nội”

Điểm khác biệt lớn nhất so với Function Calling: SERVER thực thi, client không cần biết code bên trong.



FUNCTION CALLING	MCP
• APP phải chứa hàm và tự chạy nó. • App phải có mọi thư viện, secret, quyền truy cập DB. • Muốn 3 app dùng chung → 3 bản copy của cùng logic.	• SERVER chạy — client chỉ gửi tên tool + tham số. • Secret và quyền DB nằm ở server, client không thấy. • Sửa logic tool: deploy lại server, không client nào phải sửa.

GIÁ TRỊ LỚN NHẤT

Bước 3 — TÁI SỬ DỤNG: viết một lần, dùng mọi nơi

weather_server.py

1 file · 12 dòng code

`@mcp.tool() get_weather`

Không sửa một dòng nào của server

5 client, 1 server.
Thêm client thứ 6 cũng vậy.

→ **Claude Code**
`list_tools → get_weather`
`claude mcp add weather -- python weather_server.py`

→ **Cursor**
`list_tools → get_weather`
`thêm vào mcp.json`

→ **Claude Desktop**
`list_tools → get_weather`
`claude_desktop_config.json`

→ **Gemini CLI**
`list_tools → get_weather`
`~/.gemini/settings.json`

→ **App tự viết**
`list_tools → get_weather`
`mcp.ClientSession(...)`

Chiều ngược lại cũng đúng: một client dùng được MỌI server MCP có sẵn — GitHub, Postgres, Slack, Google Drive, filesystem…

So sánh code — khai báo tool

Cùng một tool get_weather. Bên trái 20 dòng, bên phải 4 dòng.

01 — FUNCTION CALLING · viết tay

```
get_weather_declaration = types.FunctionDeclaration(
    name="get_weather",
    description="Lấy thời tiết hiện tại của một thành phố",
    parameters=types.Schema(
        type=types.Type.OBJECT,
        properties={
            "city": types.Schema(
                type=types.Type.STRING,
                description="Tên thành phố",
            )
        },
        required=["city"],
    ),
)

def get_weather(city: str) -> str:    # hàm thật, tách rời
    ...
```

Hai nơi phải khớp nhau bằng tay: schema và hàm.

02 — MCP · tự sinh

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather")

@mcp.tool()
def get_weather(city: str) -> str:
    """Lấy thời tiết hiện tại của một thành phố."""
    return f"{city}: {_MOCK_DB.get(city)}"

if __name__ == "__main__":
    mcp.run()    # mặc định chạy qua stdio
```

Một nơi duy nhất. Không thể lệch.

Code thật trong repo — server và client

weather_server.py — toàn bộ file

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("weather")

_MOCK_DB = {
    "Hanoi": "29°C, trời mưa",
    "Haiphong": "33°C, mưa rào",
    "Danang": "30°C, nhiều mây",
}

@mcp.tool()
def get_weather(city: str) -> str:
    """Lấy thời tiết hiện tại của một thành phố."""
    return f"{city}: {_MOCK_DB.get(city, '28°C')}"

if __name__ == "__main__":
    mcp.run()
```

weather_client.py — phần cốt lõi

```python
params = StdioServerParameters(
    command=sys.executable,
    args=["weather_server.py"])

async with stdio_client(params) as (read, write):
  async with ClientSession(read, write) as session:
    await session.initialize()

    # 1. KHÁM PHÁ tool (không hard-code)
    tools = await session.list_tools()
    for t in tools.tools:
        print(t.name, t.description)

    # 2. GỌI tool — SERVER thực thi
    result = await session.call_tool(
        "get_weather", {"city": "Hanoi"})
    print(result.content[0].text)
```

Chú ý: client tự khởi động server như một tiến trình con (stdio). Không cần chạy server trước, không cần API key.

LIVE DEMO · 5 PHÚT

Demo 02 — chạy thật, không cần API key

TERMINAL

```
$ cd 02-mcp-basics && python weather_client.py
```

OUTPUT

```
Tools server cung cấp:
  - get_weather: Lấy thời tiết hiện tại của một thành phố.

call_tool get_weather(city='Hanoi'):
  -> Hanoi: 29°C, trời mưa

call_tool get_weather(city='Danang'):
  -> Danang: 30°C, nhiều mây

call_tool get_weather(city='Haiphong'):
  -> Haiphong: 33°C, mưa rào
```

Điều đáng chú ý nhất

- Không có LLM nào trong demo này. Không cần API key.
- Dòng đầu tiên là kết quả list_tools — description lấy nguyên văn từ docstring.
- Giao thức MCP hoạt động độc lập với model. Model chỉ là một người dùng của nó.

Câu hỏi kiểm tra hiểu bài: Nếu bây giờ tôi thêm một tool get_air_quality vào server, client có phải sửa code không?

LUỒNG ĐẦY ĐỦ

Trong thực tế: MCP và Function Calling chạy cùng nhau

Đây là điều đang xảy ra mỗi lần bạn hỏi Claude Code một câu và nó dùng tool.

User: “Thời tiết Hà Nội?”

1. **`list_tools()` → `MCP Server`** — MCP
   Client hỏi server có tool gì. Trả về: get_weather + schema.

2. **`Gửi prompt + tool list` → `LLM`** — FUNCTION CALLING
   LLM dùng FUNCTION CALLING để chọn: “gọi get_weather(city='HN')”.

3. **`call_tool("get_weather")` → `MCP Server`** — MCP
   Server thực thi thật, trả về “HN: 29°C, mưa”.

4. **`Gửi kết quả` → `LLM tổng hợp`** — FUNCTION CALLING
   LLM viết câu trả lời: “HN 29°C, mưa, nhớ mang ô!”.

Function Calling = quyết định (bước 2, 4) · MCP = đường ống (bước 1, 3) · Chúng BỔ SUNG, không thay thế

THỰC HÀNH

Đăng ký server với AI client — làm một lần, dùng mãi

Claude Code

```
$ claude mcp add weather -- python /đường/dẫn/weather_server.py

# kiểm tra
$ claude mcp list
```

Gemini CLI · ~/.gemini/settings.json

```
"mcpServers": {
  "weather": {
    "command": "python",
    "args": ["/đường/dẫn/weather_server.py"]
  }
}
```

Sau khi đăng ký, trong Claude Code bạn chỉ cần hỏi bằng tiếng Việt:

“Thời tiết Hải Phòng hôm nay thế nào?” → Claude tự tìm thấy tool get_weather, gọi nó, rồi trả lời.

Lưu ý an toàn khi đăng ký server của người khác

Một MCP server chạy với quyền của tiến trình bạn khởi động nó. Cài server lạ = cho code lạ chạy trên máy bạn với quyền của bạn. Chỉ dùng server từ nguồn tin cậy, đọc code trước khi chạy, và giới hạn quyền (file nào, DB nào) ngay trong server.

TÓM TẮT PHẦN 1 + 2

Bảng so sánh

Tiêu chí	Function Calling	MCP
Bản chất	Khả năng của model	Giao thức client–server
Ai định nghĩa tool?	Bạn hard-code trong từng app	Server tự công bố (self-describe)
Khai báo schema	Viết tay 15–20 dòng / tool	`@mcp.tool()` → tự sinh từ type hints
Nơi thực thi tool	App của bạn tự chạy	MCP Server chạy, client điều phối
Khám phá tool	Hard-code danh sách	`list_tools()` tại runtime
Tái sử dụng	Viết lại cho mỗi app / mỗi model	Viết 1 lần, mọi MCP client dùng được
Chuẩn hoá	Mỗi provider một format	Một chuẩn chung, mở
Hệ sinh thái	Khó đóng gói chia sẻ	Hàng trăm server mã nguồn mở có sẵn
Chi phí vận hành	Không có gì thêm	Thêm tiến trình server để quản lý

Dòng cuối là điều ít ai nói: MCP không miễn phí. Với app nhỏ 1–2 tool dùng nội bộ, Function Calling thuần vẫn là lựa chọn đúng.

LAB 1 · 15 PHÚT

Tự thêm một tool vào MCP server

Mục tiêu: chứng minh bằng tay rằng client KHÔNG cần sửa gì khi server có tool mới.

Việc cần làm

1. Mở 02-mcp-basics/weather_server.py
2. Thêm tool thứ hai: get_forecast(city: str, days: int = 2)
3. Viết docstring rõ ràng — model sẽ đọc nó
4. Chạy lại python weather_client.py, xem list_tools
5. Gọi thử tool mới từ client

GỢI Ý KHUNG CODE

```python
@mcp.tool()
def get_forecast(city: str, days: int = 2) -> str:
    """Dự báo thời tiết N ngày tới cho một thành phố.

    Args:
        city: Tên thành phố (Hanoi, Danang, Haiphong)
        days: Số ngày dự báo, 1-3 (mặc định 2)
    """
    # TODO: trả về chuỗi dự báo của bạn
```

Xong sớm? Thử tiếp:

Đăng ký server vào Claude Code và hỏi bằng tiếng Việt “dự báo Đà Nẵng 3 ngày tới”.

Điều cần quan sát: bạn KHÔNG sửa weather_client.py, nhưng nó vẫn thấy tool mới. Đó là self-describing.

PHẦN 3

MCP trong Production

Ba khoảng trống giữa “chạy trên máy tôi” và “chạy cho cả công ty”.
Code mẫu: 03-production/

Từ demo tới production: ba khoảng trống

Demo dùng stdio trên cùng một máy, ai chạy cũng được, chỉ một phiên bản tool. Production không như vậy.

SECURITY

Ai được gọi tool?

Server phục vụ nhiều client qua mạng → phải xác thực và phân quyền.

```
Bearer token · OAuth · scopes
```

REGISTRY

Agent biết tool nào tồn tại?

Hàng chục server, hàng trăm tool. Không thể hard-code trong agent.

```
Tìm tool theo tag / capability
```

VERSIONING

Đổi tool mà không phá client cũ?

Client cũ vẫn đang chạy production. Đổi schema là phá họ.

```
v1 + v2 song song · deprecation
```

Ba file trong 03-production/ giải quyết đúng ba vấn đề này: auth_server.py · registry_client.py · versioned_server.py

SECURITY

3a · Authentication — stdio thì không cần, HTTP thì bắt buộc

Tầng	Demo (stdio)	Production (HTTP)
Transport	stdio — tiến trình con trên cùng máy	**Streamable HTTP — qua mạng**
Ai gọi được?	Chỉ người đã đăng nhập máy đó	**Bất cứ ai tới được URL → phải xác thực**
Auth	Không cần	Bearer token · OAuth · mTLS
Phạm vi truy cập	Toàn bộ tool	Scopes giới hạn theo từng client
Ghi vết	Không có	Log ai gọi tool nào, lúc nào

Luồng xác thực



Điểm hay: SDK xử lý ở tầng transport — hàm get_weather() không có một dòng nào về auth.

3a · Code — TokenVerifier ở server, header ở client

Server

```python
class StaticTokenVerifier(TokenVerifier):
    async def verify_token(self, token: str) -> AccessToken | None:
        client_id = VALID_TOKENS.get(token)
        if client_id is None:
            return None            # -> 401 / 403
        return AccessToken(token=token, client_id=client_id,
                           scopes=["weather:read"])

mcp = FastMCP(
    "weather-secure", host="0.0.0.0", port=8000,
    auth=AuthSettings(
        issuer_url="http://localhost:8000",
        resource_server_url="http://localhost:8000"),
    token_verifier=StaticTokenVerifier(),
)

mcp.run(transport="streamable-http")    # không còn stdio
```

Client

```python
http_client = httpx.AsyncClient(
    headers={"Authorization":
             f"Bearer {TOKEN}"})

async with streamable_http_client(
        SERVER_URL,
        http_client=http_client) as (r, w, _):
    async with ClientSession(r, w) as session:
        await session.initialize()
        await session.call_tool(
            "get_weather", {"city": "Hanoi"})
```

Production thay gì?

- VALID_TOKENS dict → JWT verify, OAuth introspection, hoặc Keycloak / Auth0 / Google IAM.
- Token trong code → biến môi trường hoặc secret manager.
- scopes → kiểm tra thật: client này được gọi tool nào.

Cách chạy: terminal 1 `python auth_server.py` · terminal 2 `python auth_client.py`

DISCOVERY

3b · Tool Registry — agent tự tìm tool thay vì hard-code

Với 30 server và 200 tool, agent không thể biết trước cái nào ở đâu. Nó phải tra danh mục.



REGISTRY.JSON — MỘT ENTRY

```
"get_weather_v2": {
  "description": "Lấy thời tiết chi tiết — JSON,
                  hỗ trợ forecast và đơn vị đo",
  "parameters": {
    "city":  {"type": "string",  "required": true},
    "include_forecast": {"type": "boolean",
                         "default": false},
    "units": {"type": "string", "default": "celsius"}
  },
  "tags": ["weather", "forecast", "detailed"],
  "server": "weather-v2",
  "version": "2.0.0",
  "deprecated": false
}
```

Registry là TOOL-CENTRIC: đơn vị khám phá là tool (tag, description, parameters), không phải server. Agent tìm theo năng lực cần, không theo địa chỉ.

3b · Code — search, best_match, và tự chọn transport

TRA CỨU

```python
def search(self, *, tag=None, keyword=None) -> list[dict]:
    results = []
    for tool_name, cfg in self.tools.items():
        if tag and tag in cfg.get("tags", []):        results.append(...)
        if keyword and keyword.lower() in cfg["description"].lower(): ...
    return results

def best_match(self, *, tag=None, keyword=None) -> dict:
    results = self.search(tag=tag, keyword=keyword)
    active = [r for r in results if not r["deprecated"]]
    return max(active or results, key=lambda r: r["version"])
```

TỰ CHỌN TRANSPORT + AUTH

```python
if server["transport"] == "stdio":
    params = StdioServerParameters(command=sys.executable,
                                   args=server["args"])
    async with stdio_client(params) as (read, write): ...

elif server["transport"] == "streamable-http":
    if server.get("auth", {}).get("type") == "bearer":
        token = os.environ[server["auth"]["token_env"]]
        headers["Authorization"] = f"Bearer {token}"
    async with httpx.AsyncClient(headers=headers) as http: ...
```

Hạn chế của bản demo

- max() trên chuỗi version: "10.0.0" < "9.0.0". Production dùng semver parse thật.
- Không health check — server chết vẫn được chọn.
- Không cache: mỗi lần gọi là mở kết nối mới.
- Tìm theo tag/keyword, chưa phải semantic search.

3b · Hard-code so với Registry

	Hard-code (demo)	Tool Registry (production)
Agent biết tool nào?	Chỉ tool được code sẵn	Tất cả tool trong registry
Cách tìm tool	Theo tên cố định	Theo tag, keyword, capability
Thêm tool mới	**Sửa code agent + deploy lại**	**Thêm một entry vào registry**
Ai chọn tool	Developer quyết định lúc viết code	Agent tự chọn best match lúc chạy
Đổi server phía sau	Sửa code agent	Đổi mục "server" trong registry

Vì sao đáng làm

- Thêm năng lực cho agent mà không deploy lại agent.
- Team nền tảng quản registry, team nghiệp vụ chỉ đăng ký tool.
- Một chỗ duy nhất để audit: ai có tool gì.

Cái giá phải trả

- Registry trở thành single point of failure — phải HA và cache.
- Agent tự chọn tool = hành vi khó đoán hơn, cần log kỹ.
- Quá nhiều tool làm model chọn kém đi. Lọc trước khi đưa cho model.

BACKWARD COMPATIBILITY

3c · Versioning — đổi tool mà không phá client đang chạy

Tình huống

Server v1 có get_weather(city) trả về chuỗi “Hanoi: 29°C, trời mưa”. Giờ bạn muốn trả JSON chi tiết và thêm tham số include_forecast. Nếu sửa trực tiếp → mọi client đang parse chuỗi đó sẽ vỡ, ngay lúc bạn deploy.

1

Tool mới song song

get_weather_v2 tồn tại BÊN CẠNH get_weather. Không xoá tool cũ, chỉ đánh dấu deprecated.

2

Tham số optional có default

include_forecast=False, units="celsius". Client cũ gọi get_weather_v2(city="Hanoi") vẫn đúng.

3

Server metadata công bố

Resource server://info nói rõ version, tool nào deprecated, và hướng dẫn chuyển đổi.

Server v2 sau khi áp dụng cả ba

```
├── get_weather(city)                          ← v1, deprecated nhưng VẪN HOẠT ĐỘNG
├── get_weather_v2(city, include_forecast, units)   ← v2, đầy đủ tính năng
└── resource server://info                       ← version + migration guide
```

3c · Code — hai tool song song + resource metadata

HAI TOOL CÙNG TỒN TẠI

```python
@mcp.tool()
def get_weather(city: str) -> str:
    """[v1] Lấy thời tiết hiện tại — trả chuỗi đơn giản.
       Deprecated, dùng get_weather_v2."""
    return f"{city}: {data['temp']}°C, {data['condition']}"

@mcp.tool()
def get_weather_v2(city: str,
                   include_forecast: bool = False,
                   units: str = "celsius") -> str:
    """[v2] Lấy thời tiết chi tiết — JSON, hỗ trợ forecast."""
    if units == "fahrenheit": temp = round(temp * 9/5 + 32, 1)
    result = {"api_version": "2.0", "city": city, "temp": temp, ...}
    if include_forecast: result["forecast"] = data["forecast"]
    return json.dumps(result, ensure_ascii=False)
```

METADATA CHO CLIENT ĐỌC

```python
@mcp.resource("server://info")
def server_info() -> str:
    return json.dumps({
      "name": "weather-v2",
      "version": "2.0.0",
      "deprecated_tools": ["get_weather"],
      "migration_guide":
        "Chuyển sang get_weather_v2. 'city'
         giữ nguyên, thêm include_forecast
         và units.",
    })
```

Client thông minh làm gì

Đọc server://info TRƯỚC → biết tool nào deprecated → tự chọn v2 nếu có, fallback v1 nếu server còn cũ. Xem versioned_client.py: read_resource → list_tools → call_tool.

TỔNG KẾT PHẦN 3

Checklist mang về: Dev → Production

Khía cạnh	Dev / Demo	Production
Transport	stdio, cùng máy	Streamable HTTP / SSE qua mạng
Auth	Không có	Bearer token, OAuth, mTLS + scopes
Secret	Hard-code trong file	Biến môi trường / secret manager
Discovery	Hard-code tool và server	Tool Registry, agent tìm theo task
Versioning	Một tool duy nhất	v1 + v2 song song, deprecation notice
Health	Không có	Health check, retry, circuit breaker
Logging	`print()`	Structured logging, tracing (OpenTelemetry)
Giới hạn	Không	Rate limit, timeout, quota theo client
Kiểm tra input	Tin model	Validate mọi tham số model sinh ra

PHẦN 4

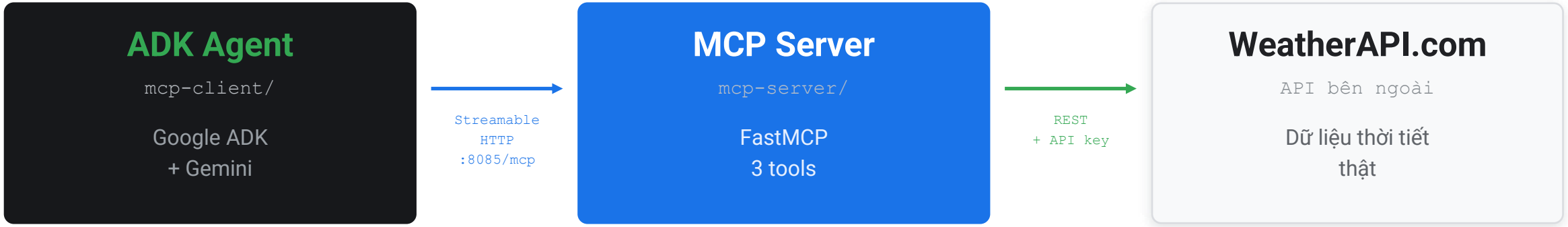
Thực chiến & Tổng kết

Một agent hoàn chỉnh: framework làm client, MCP server gọi API thật, deploy container.
Code mẫu: 04-lab/

04-LAB/

Kiến trúc một agent thật

Khác hai demo trước: có framework làm client, có API thật, có container để deploy.



Ba tool server công bố

`get_current_weather(city)`	Thời tiết hiện tại của một thành phố
`get_forecast(city, days=3)`	Dự báo 1–3 ngày (giới hạn gói free)
`health_check()`	Kiểm tra server sống — bắt buộc khi deploy

Điểm mới so với 02 và 03

- Tool là async def — gọi HTTP ra ngoài không chặn server.
- API key đọc từ biến môi trường WEATHERAPI_KEY.
- Có Dockerfile → deploy được lên Cloud Run.
- health_check() để nền tảng biết container đã sẵn sàng.

GOOGLE ADK

Framework làm gì? Nó chính là MCP Client của bạn

So với bài 02 nơi ta tự viết vòng lặp: framework làm hộ toàn bộ điều phối.

1. **KẾT NỐI tới server**
2. **KHÁM PHÁ tool tự động**
3. **TRUYỀN tool cho LLM**
4. **ĐIỀU PHỐI vòng lặp Function Calling**
5. **CUNG CẤP giao diện**

AGENT.PY — GẦN NHƯ TOÀN BỘ

```python
from google.adk import Agent
from google.adk.tools.mcp_tool.mcp_toolset import (
    McpToolset, StreamableHTTPConnectionParams)

connection_params = StreamableHTTPConnectionParams(
    url="http://localhost:8085/mcp", timeout=30.0)

weather_tools = McpToolset(
    connection_params=connection_params)

root_agent = Agent(
    name="weather_agent",
    model="gemini-2.5-flash",
    tools=[weather_tools],    # <- không viết vòng lặp
)
```

Bài 02 ta tự viết list_tools + call_tool + vòng lặp. Ở đây 15 dòng thay cho tất cả — vì MCP là chuẩn, framework mới làm hộ được.

Năm việc framework làm hộ — chi tiết

1

KẾT NỐI tới MCP server

Mở kết nối Streamable HTTP, giữ session, tự retry khi timeout (Cloud Run có cold start).

2

KHÁM PHÁ tool tự động

McpToolset gọi list_tools ngay khi khởi động. Server thêm tool mới → agent thấy sau khi restart, không sửa code.

3

CHUYỂN ĐỔI schema cho model

Lấy schema MCP → dịch sang format Function Calling của Gemini. ĐÂY CHÍNH LÀ CHỖ HAI KHÁI NIỆM GẶP NHAU: MCP nằm trên, Function Calling nằm dưới.

4

ĐIỀU PHỐI vòng lặp

Gọi model, đọc function_calls, gọi call_tool, đưa kết quả về, lặp đến khi model không cần tool nữa.

5

GIAO DIỆN + phiên hội thoại

adk web cho UI chat, lưu lịch sử hội thoại vào session, không phải tự quản contents như bài 01.

DEPLOY

Đưa MCP server lên mạng — container hoá

DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY pyproject.toml ./
RUN pip install --no-cache-dir -e .
COPY weather.py ./
EXPOSE 8080
ENV PYTHONUNBUFFERED=1
CMD ["python", "weather.py"]
```

MỘT FILE, HAI CHẾ ĐỘ CHẠY

```
port = int(os.getenv("PORT", 8085))
mcp = FastMCP("weather", host="0.0.0.0", port=port)

# Cloud Run set biến PORT -> chạy HTTP
# Không có PORT + có tty -> cũng chạy HTTP
# Còn lại -> stdio cho client local
if is_cloud_run or is_standalone:
    mcp.run(transport="streamable-http")
else:
    mcp.run()
```

host="0.0.0.0"	Bắt buộc trong container. Dùng 127.0.0.1 thì bên ngoài không vào được — lỗi số 1 khi deploy.
Đọc PORT từ env	Cloud Run / Heroku / Fly tự cấp port. Hard-code port là deploy thất bại.
Auth ngay từ đầu	Server công khai không auth = ai cũng gọi tool của bạn. Áp phần 3a trước khi mở ra Internet.
health_check()	Nền tảng dùng nó để biết container sẵn sàng và để tự khởi động lại khi treo.

HƯỚNG DẪN QUYẾT ĐỊNH

Khi nào dùng cái nào?

DÙNG FUNCTION CALLING THUẦN KHI…

- App chỉ có 1–3 tool, gắn chặt với chính app đó.
- Không có kế hoạch chia sẻ tool cho app/team khác.
- Bạn khoá vào một provider và chấp nhận điều đó.
- Muốn ít thành phần vận hành nhất có thể.
- Tool cực nhẹ (tính toán thuần) — thêm một tiến trình server là quá đáng.

DÙNG MCP KHI…

- Muốn tool dùng lại được trên nhiều AI client (IDE, agent, chatbot).
- Cần tách logic tool khỏi app — hai team, hai nhịp deploy.
- Secret / quyền DB không nên phát cho từng app.
- Muốn dùng lại server có sẵn: GitHub, Postgres, Slack, Drive…
- Đang xây hệ sinh thái tích hợp nội bộ dài hạn.

Và câu trả lời hay gặp nhất trong thực tế: cả hai

Bạn không chọn giữa hai thứ ở hai tầng khác nhau. Trong một agent production điển hình: MCP để cắm tool vào, Function Calling là cách model quyết định gọi tool nào. Nếu có sẵn MCP server phù hợp — cắm vào. Nếu tool chỉ dùng cho một app và không bao giờ đi đâu — Function Calling thuần là đủ.

KINH NGHIỆM

Bảy sai lầm thường gặp

1	**Tưởng model tự chạy hàm**	Model chỉ sinh JSON. Mọi thực thi là code của bạn — nên mọi kiểm soát cũng ở đó.
2	**Docstring / description viết mờ**	Đây là nguyên nhân số 1 của “model gọi sai tool”. Viết rõ tool làm gì và KHI NÀO nên dùng.
3	**Không validate tham số model sinh**	Tham số từ model là input không tin cậy. Với query_db thì model đang viết SQL cho bạn chạy.
4	**Nhồi hàng trăm tool cho model**	Càng nhiều tool, model càng chọn kém và càng tốn token. Lọc theo ngữ cảnh trước khi đưa vào.
5	**Deploy server public không auth**	stdio không cần auth nên dễ quên khi chuyển sang HTTP. Ai tới được URL là gọi được tool.
6	**Sửa schema tool đang chạy**	Client cũ vỡ ngay lúc deploy. Thêm tool mới song song, đánh dấu cũ là deprecated.
7	**Cài MCP server lạ không đọc code**	Server chạy bằng quyền của bạn trên máy bạn. Chỉ dùng nguồn tin cậy.

TỔNG KẾT

1

Function Calling là khả năng của model

Model đọc schema, chọn tool, sinh JSON tham số. Nó không bao giờ tự thực thi.

2

MCP là giao thức, không phải model

Chuẩn client–server để khám phá và gọi tool. Chạy được không cần LLM nào — như demo 02.

3

MCP dùng Function Calling bên dưới

Framework lấy schema từ MCP và đưa vào cơ chế Function Calling của model. Hai tầng, không cạnh tranh.

4

@mcp.tool() xoá boilerplate

Schema tự sinh từ type hints + docstring. Một nguồn sự thật, không lệch được.

5

Production = Auth + Registry + Versioning

Cộng thêm health check, logging, rate limit và validate input. Đó là khoảng cách giữa demo và hệ thống.

Bài tập về nhà & tài nguyên

Ba bài tập, tăng dần

DỄ **Viết MCP server cho công việc của bạn**

Chọn một việc bạn làm tay hàng ngày (tra đơn hàng, đọc log, tính toán nội bộ). Bọc nó thành 1–2 tool, đăng ký vào Claude Code, dùng thử.

TRUNG BÌNH **Thêm auth cho server đó**

Chuyển từ stdio sang streamable-http, thêm TokenVerifier, gọi từ máy khác trong mạng LAN. Thử token sai để thấy 401/403.

KHÓ **Versioning trên tool thật**

Đổi format trả về của một tool đang dùng — nhưng giữ client cũ chạy được. Thêm resource server://info và một client đọc metadata trước khi gọi.

Tài nguyên

Repo buổi học

`Github — 4 module`

Spec MCP

`modelcontextprotocol.io`

Python SDK

`modelcontextprotocol/python-sdk`

Server có sẵn

`modelcontextprotocol/servers`

Function calling

`ai.google.dev`

Gợi ý: bài “Dễ” là bài quan trọng nhất. Một tool thật bạn dùng hàng ngày dạy bạn nhiều hơn mười tool ví dụ.

Cảm ơn cả lớp!

Nguyen Hai Truong







/truongnh1992